

ProbOS — Month 2, Week 5

Particle Filter Core

Nisong Monyimba

July 3, 2026

Week 5 goal

Build `ParticleFilter` – ProbOS’s first sequential Bayesian inference tool – and validate it against an exact closed-form solution (the Kalman filter on a linear-Gaussian model) before ever touching `BatteryModel2Cell`.

Day-by-day plan

Day 1 – Algorithm design

- Bootstrap (SIR) particle filter: predict / update / resample
- Log-space weights throughout, exponentiate only at the point of use – same underflow-avoidance discipline as `Distribution.log_pdf()` from Week 1
- `python/src/particle_filter.py` skeleton: `ParticleFilter`, `PFResult`

Day 2 – predict/update/resample implementation

- `predict()` reuses `Model.forward_batch()` unmodified – zero changes to the Week 2 `Model` ABC
- `update()`: log-space reweighting via a caller-supplied log-likelihood function
- `resample()`: systematic (low-variance) resampling
- Effective sample size (ESS) diagnostic and resample-when-low trigger

Day 3 – Kalman filter ground truth

- Implement the exact 1D linear-Gaussian Kalman filter recursion directly (not imported from any library) as validation ground truth
- `RandomWalkModel`: a minimal linear-Gaussian test model

Day 4 – Validation against Kalman ground truth

- Posterior mean RMSE against exact Kalman filter mean
- Posterior std within a sane range of exact Kalman filter std
- Confirm Monte Carlo error shrinks from $N = 100$ to $N = 2000$ (convergence, not just agreement at one N)

Day 5 – Unit tests + hardening

- `python/tests/test_particle_filter.py`: construction, mechanics (predict/update/resample in isolation), and the full Kalman ground-truth validation suite
- mypy strict, ruff clean

Day 6-7 – Study guide + commit

- `docs/study/study_guide.md` entries: Naesseth et al. (2019), Chopin & Papaspiliopoulos (2020), Sarkka (2013)
- Full test suite, CI verification

What was actually accomplished (retrospective)

- `ParticleFilter` implemented exactly as planned
- 20 tests: unit-level mechanics plus the critical Kalman ground truth validation
- Genuine mathematical proof, not a smoke test: posterior mean and std converge to the exact closed-form Kalman filter solution, with Monte Carlo error provably shrinking as N grows
- This is the same “validate against a known analytical case before touching the real model” discipline established in Week 2 (Kim 2007) applied to a new class of algorithm (inference, not forward simulation)

Numbers at Week 5 close

- Python tests: 284/284 passing (264 Month 1 + 20 new)
- C++ tests: 13/13 passing
- mypy strict: 0 errors
- ruff: 0 warnings

Carried into Week 6

`ParticleFilter` is validated and ready to be applied to `BatteryModel2Cell` with synthetic noisy observations in a later week. Week 6 turns to a different concern: binding the Week 4 C++ kernel into the Python package via `pybind11`, so the two implementations stop being parallel and disconnected.